

Documentation technique d'EcoRide

1) Réflexion initiale technique de l'application

Choix des technologies :

Front-end :

- **HTML5** : pour structurer les pages, améliorer le SEO et garantir une accessibilité avec la sémantique.
- **CSS3** : pour gérer l'apparence, le design, les couleurs, la disposition des éléments.
- **Tailwind CSS** : Framework CSS pour accélérer le développement, réduire le code CSS et garantir une interface cohérente.
- **Bibliothèques UI** (MerakiUI, HyperUI, PrelineIcon) et **icônes** (Heroicons) : pour s'inspirer, un design harmonieux et une personnalisation facile.
- **JavaScript** : rend l'interface dynamique, permet les animations, le DOM dynamique, la communication avec des API (cartes, graphique, back end) et améliore la fluidité et l'interactivité des pages.

Back-end :

- **PHP (POO, MVC) avec PDO**
 - **POO** : facilite la réutilisation, la sécurité et la maintenance du code
 - **MVC** : structure le code pour séparer la logique métier, la vue et les contrôleurs
 - **PDO** : sécurise les requêtes SQL et protège contre les injections SQL
- **Composer** : gestion des dépendances (PHPMailer, autoload...), mise à jour des bibliothèques facilement

Base de données :

- **MySQL (SQL)** : base de données relationnelle populaire, gratuite, open source, performante et compatible avec PHP
- **MongoDB (NoSQL)** : base NoSQL pour les données non structurées, flexible et rapide

Architecture et infrastructure :

- **Docker** : architecture en micro-services pour séparer les logiciels, sécuriser l'environnement et garantir l'harmonie entre les systèmes d'exploitation
- **Nginx** : serveur web performant et sécurisé.

Intégration avec d'autres services et API :

- **Leaflet.js** : cartographie interactive pour visualiser les trajets et les points géographiques, léger, open source
- **PHPMailer** : envoi automatique de mail aux utilisateurs, sécurisé, gestion du HTML, gestion des complexités
- **Nominatim** : vérification et validation des villes saisies par les utilisateurs, open source
- **HeiGIT** : calcul d'itinéraires optimisés entre deux points, open source
- **Chart.js** : création simple de graphiques, open source et personnalisation

Gestion de projet :

- **Notion** : organisation des tâches, suivi de projet et documentation centralisée, il est flexible, vue native kanban et personnalisable

Ressources graphiques :

- **Gemini, Freepik, Unsplash** : images et illustrations libres de droits pour enrichir l'interface et améliorer l'expérience utilisateur

Déploiement :

- **AlwaysData** : permet de mettre le site en ligne (production), serveur en Europe et hébergement écologique

2) Configuration de l'environnement de travail

J'ai travaillé sur un ordinateur sous **Windows 11**.

Pour le développement du projet, plusieurs outils ont été installés et configurés :

Pour l'éditeur de code j'ai choisi **Visual Studio Code** car il est gratuit, open source, très populaire et dispose d'une grande variété d'extensions.

Il offre aussi un terminal intégré et une prise en charge de nombreux langages.

J'ai installé **Git** afin de versionner le projet et gérer les différentes branches.

Les commandes Git permettent de créer un espace de stockage (git init), créer plusieurs branches pour les fonctionnalités, de les fusionner et de synchroniser le code avec un dépôt distant.

J'ai créé un dépôt **GitHub** et lié au dépôt local. Avec la commande « `git remote add origin https://github.com/votre-nom-utilisateur/mon-nouveau-projet.git` »

Un fichier **.gitignore** a été ajouté pour exclure les fichiers sensibles et volumineux (.env ou vendor).

J'ai créé un fichier **.env** et **.env.example** pour gérer et expliquer les variables d'environnement à définir. Le fichier .env contient les informations sensibles concernant la configuration des identifiants MySQL et MongoDB, de ports de Nginx et de paramètres de PHPMailer.

J'ai installé **Docker Desktop** avec Docker Engine, Docker CLI et Docker Compose.

L'utilisation de Docker permet d'isoler l'environnement de développement et de garantir que l'application fonctionnera de la même manière sur toutes les machines.

Un fichier « **docker-compose.yml** » a été créé afin de définir et orchestrer les services nécessaires :

- **MySQL** (SGBDR gratuit, open source, performant et compatible avec PHP)
- **MongoDB** (base NoSQL, flexible et rapide)
- **PHP-FPM** (back end PHP, relié à un **Dockerfile** personnalisé)
- **Nginx** (serveur web performant et sécurisé)

Les images utilisées sont basées sur des versions slim ou alpine pour la sécurité, la performance et le minimalisme. Les volumes ont été configurés pour persister les données.

J'ai ensuite configuré le **Dockerfile** de **PHP**, j'ai installé des extensions PHP comme le PDO pour communiquer avec MySQL et l'extension de MongoDB pour permettre la connexion à la base NoSQL. Composer a été installé pour gérer les dépendances (PHPMailer, MongoDB et l'autoloading)

Un **Dockerfile** pour **Nginx** a été créé, un fichier **nginx.conf** aussi. Ce fichier définit le répertoire racine (public), limite la taille des fichiers uploadés, définit la page d'accueil par défaut. Interdit l'accès aux fichiers cachés, définit l'alias pour le site de l'administrateur.

J'ai structuré mon projet avec le design pattern **MVC** pour séparer les différents logiques métier et faciliter la lecture et la maintenance.

J'ai créé un fichier **composer.json** pour configurer l'autoload et les différentes dépendances.

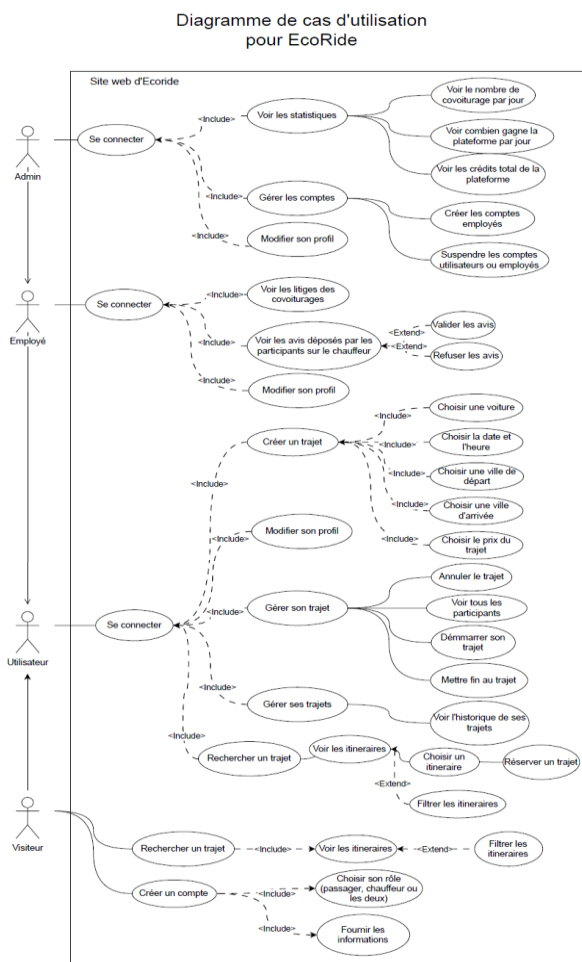
Après configuration, la commande suivante a permis de lancer l'environnement : **docker-compose up --build -d**. Ensuite, j'ai exécuté composer pour installer les dépendances et générer l'autoload avec : **docker-compose exec php-fpm composer dump-autoload**
A la suite de cela, un dossier **vendor** a été créé.

J'ai créé mes tables (roles, users, cars, preferences, car_sharing, reservations, feedbacks et company), grâce à des requêtes SQL comme : **CREATE TABLE roles(colonnes de la table);**

Puis j'ai inséré les rôles dans la table **roles** avec la commande : **INSERT INTO roles(noms des tables) VALUES (valeur) ;**

J'ai créé deux classes une **MySQL** pour se connecter à la base de données MySQL avec PDO en utilisant les variables d'environnement et une classe **MongoDb** pour se connecter à MongoDB via l'URI et le client MongoDB.

3) Diagramme de cas d'utilisation



Le **diagramme de cas d'utilisation** sert à représenter comment les différents **acteurs** interagissent avec l'application.

Les acteurs sont représentés par un petit bonhomme, le système(l'application) par un rectangle et les **cas d'utilisation** par des ovales reliés par des lignes.

On utilise *include* quand une action utilise toujours une autre et *extend* quand une action peut parfois s'ajouter à une autre.

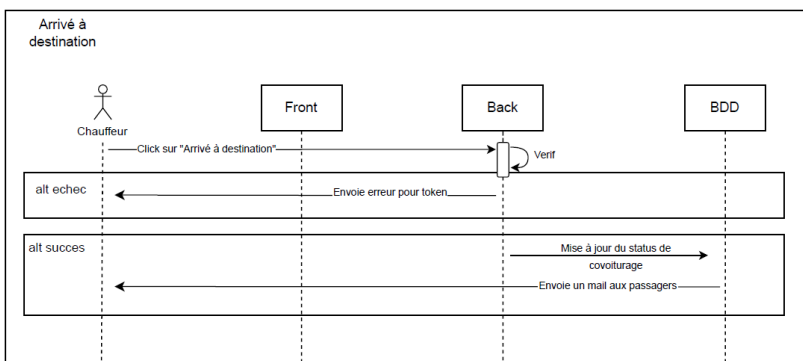
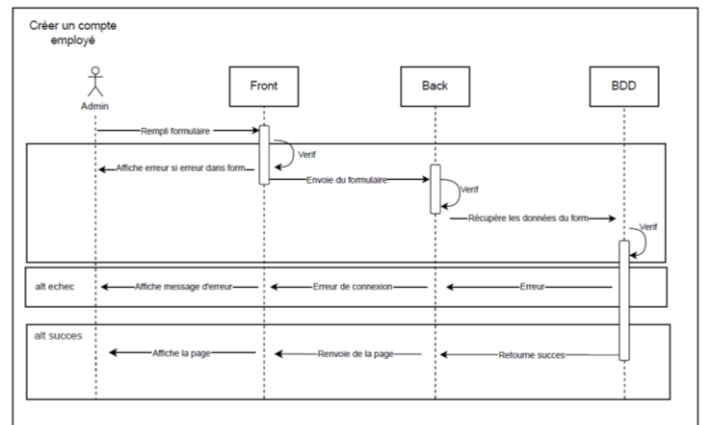
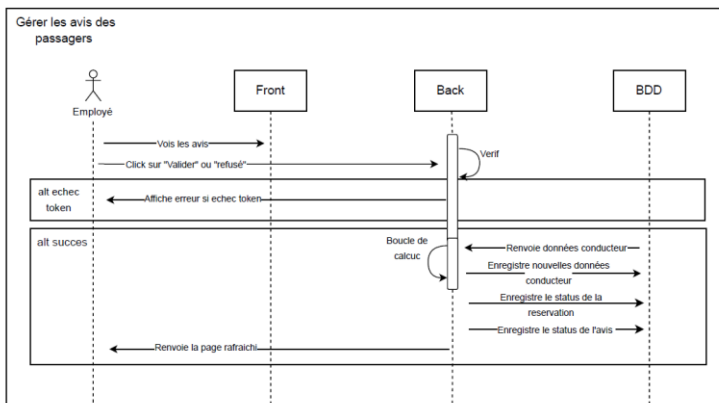
Les acteurs du projet sont l'application, les visiteurs, les utilisateurs, les employés et l'administrateur. Quelques exemples d'action gérer ses trajets, gérer les comptes et rechercher un trajet.

4) Diagramme de séquences

Le **diagramme de séquence** sert à représenter l'**enchaînement** des **interactions** entre différents **objets** ou **acteurs** d'un système dans le temps.

Les acteurs et objets sont placés en haut du diagramme. Il peut s'agir d'un utilisateur ou d'un système. Les **lignes de vie** sont des lignes verticales situées sous chaque acteur ou objet, représentant leur existence dans le temps. Les **messages** et **interactions** sont représentés par des flèches horizontales. Les **actions** sont représentées par des rectangles et indique que l'objet est actif et exécute une opération.

Diagramme de séquences pour
EcoRide



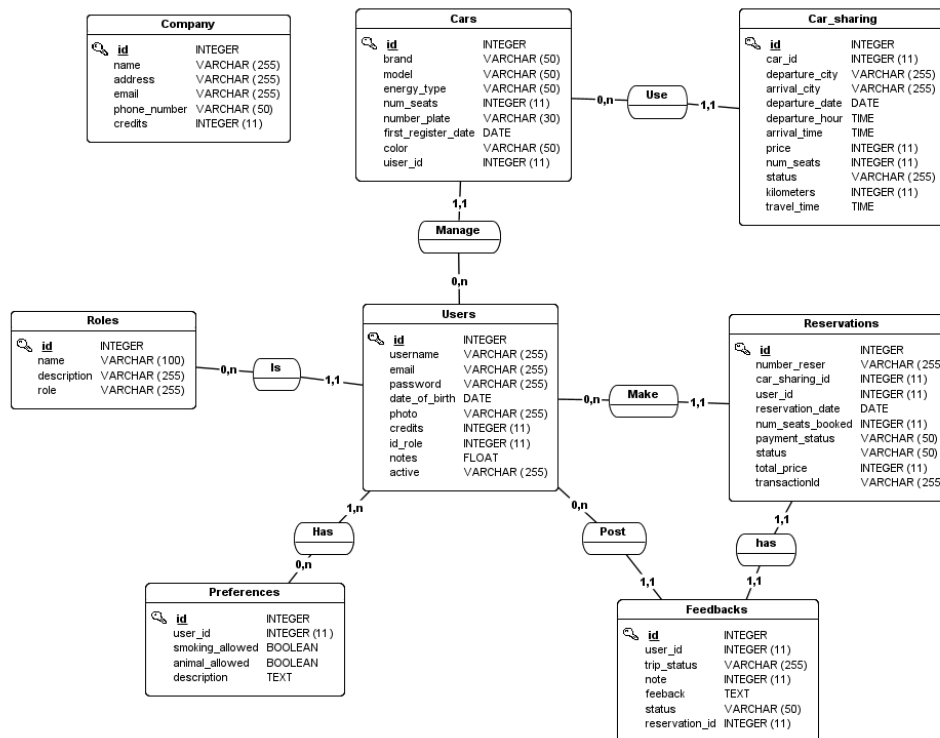
5) Modèle conceptuel de données

Le **Modèle Conceptuel de Données** (MCD) est un schéma qui permet de représenter les **données** et leurs **relations** dans un système.

Les **entités** correspondent généralement aux tables représentées par des rectangles. Elles possèdent des **attributs** qui représentent leurs propriétés.

Les **associations** sont représentées par des ovales et indiquent les liens entre les entités.

Les **cardinalités** définissent le nombre **minimum** et **maximum** de **relations** possibles entre deux entités(ex : 0,n).



6) Documentation du déploiement de l'application

Je me suis créé un compte **Alwaysdata**, puis j'ai choisi un abonnement de serveur mutualisé qui prend en charge **MongoDB**. J'ai ensuite défini un nom pour le site, qui sera utilisé pour l'adresse de sous-domaine (<http://nom-du-site.alwaysdata.net>), ainsi qu'un mot de passe fort.

J'ai téléchargé **FileZilla** afin de pouvoir transférer les dossiers et fichiers du projet via **FTP**. Dans FileZilla, j'ai renseigné les informations FTP disponibles dans mon espace Alwaysdata (nom d'hôte, identifiants FTP), puis j'ai cliqué sur connexion rapide. Une fois connecté au serveur, je suis allé dans mon dossier racine et j'ai transféré les fichiers du projet.

Ensuite, je me suis rendu dans l'onglet **Web > Sites** pour modifier les configurations du site par défaut qui avait été créé.

J'ai choisi **PHP 8.3** (utilisé en développement dans Docker) afin d'éviter les conflits. J'ai configuré le répertoire racine, php.ini et les variables d'environnement et j'ai forcé toutes les requêtes en HTTPS pour plus de sécurité (grâce au certificat SSL fourni par Alwaysdata).

Leur serveur prend en charge **PHP Apache** mais pas **Nginx** donc j'ai dû adapter mes configurations **Nginx** vers **Apache**, en utilisant un fichier **.htaccess** et des directives **VirtualHost** pour les alias et réglages spécifiques.

Je suis ensuite allé dans la section **Bases de données MySQL**, où j'ai créé une base de données avec son nom et ses permissions, puis j'ai configuré un mot de passe pour renforcer la sécurité.

J'ai importé mes tables via **phpMyAdmin**, puis j'ai mis à jour les variables d'environnement du projet pour correspondre à la base de données du serveur.

Ensuite, via mon terminal, je me suis connecté en **SSH** pour installer **MongoDB**, car Alwaysdata ne le fournit pas directement.

A l'aide des lignes de commande, j'ai téléchargé MongoDB, créé un répertoire pour stocker les données et les logs (data et log), et extrait l'archive compressée puis j'ai configuré un service dans Services avancé.

J'ai renseigné la commande de démarrage, le port utilisé, ainsi que le répertoire associé.

Commande pour créer un dossier : **mkdir mongodb**

Commande pour aller au dossier : **cd mongodb**

Commande renommer fichier : **mv mongodb-linux-x86_64-debian12-8.2.0 bin**

Commande pour installer mongodb : **wget https://fastdl.mongodb.org/linux/mongodb-linux-x86_64-debian12-8.2.0.tgz**

Et extraire le fichier compressé : **tar -zxvf mongodb-linux-x86_64-debian12-8.2.0.tgz**

Commande de démarrage : **./bin/mongod --dbpath ./data/ --logpath ./log/mongo.log --ipv6 --bind_ip_all --port=...** trouvé dans la documentation.

Pour PHP, j'ai installé l'extension MongoDB avec la commande : **ad_install_pecl mongodb**

Ensuite, j'ai ajouté la ligne suivante dans **php.ini** afin de charger l'extension :

extension=/home/nom-du-sous-domain/mongodb-8.3.so pour activer la configuration Puis

j'ai actualisé l'autoload avec Composer : **composer dump-autoload**